Internship Report On

“NTPWEdit Clone - Windows Password Management Tool”

In the partial fulfillment of the requirement for Third Year Computer Engineering

Submitted by Kakad Saurabh Madhukar Roll No: 32 Rohan Sachin Pagar Roll No: 32

Under the Guidance of Prof. P. P. Kakade

DEPARTMENT OF COMPUTER ENGINEERING

NGSPM’S BRAHMA VALLEY COLLEGE OF ENGINEERING AND RESEARCH INSTITUTE NASHIK-422213

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE 2025-2026

ACKNOWLEDGEMENT

It is a great pleasure for us to express our deep sense of gratitude to all those who have helped us directly and indirectly in the successful completion of this internship project.

We would like to express our special thanks of gratitude to our guide, **Prof. P. P. Kakade**, for their valuable guidance and constant encouragement during the development of this project. Their insights and technical expertise were instrumental in overcoming the various challenges we faced.

We also express our sincere thanks to the **H.O.D. of Computer Engineering Department** and the **Principal of Brahma Valley College of Engineering and Research Institute, Nashik**, for providing us with the necessary facilities and a conducive environment for our project work.

Finally, we would like to thank our parents and friends for their continuous support and motivation.

Kakad Saurabh Madhukar Rohan Sachin Pagar

ABSTRACT

The Security Accounts Manager (SAM) hive is a critical component of the Windows NT operating system family, storing sensitive user account information, including password hashes and group memberships. Management of these accounts typically requires the operating system to be active; however, in scenarios involving system lockouts or forensic investigations, offline access is required.

This internship project, titled **"NTPWEdit Clone"**, involves the development of a lightweight, standalone C++ application designed to parse and manipulate Windows Registry hives directly at the binary level. By bypassing the Windows Registry APIs, the tool can operate on SAM files located on non-booted drives, such as those accessed via Windows Recovery Environment (WinRE) or a bootable Linux/Live PE environment.

The project demonstrates a modular approach to system-level programming, featuring a custom registry hive parsing engine, a SAM-specific data manager, and a native Win32 user interface. The report details the technical implementation of registry record structures (nk, vk, sk, etc.), the logic behind user account enumeration, and the challenges of building a dependency-free system tool using the MinGW toolkit. The result is a functional utility that automates SAM detection and provides a clear interface for account visualization and planned password management.

INDEX

Sr. No.	Content	Page No.
1	**INTRODUCTION**	**1**
1.1	Project Overview	1
1.2	Problem Statement	2
1.3	Objectives of the Project	3
1.4	Scope of the Project	4
1.5	Methodology	5
2	**LITERATURE SURVEY / BACKGROUND**	**7**
2.1	Evolution of the Windows Registry	7
2.2	Windows Security Architecture History	8
2.3	The Security Accounts Manager (SAM) Logic	10
2.4	NTLM vs. Kerberos Authentication	12
3	**REGISTRY HIVE FORMAT DEEP DIVE**	**14**
3.1	Structural Overview	14
3.2	The REGF Header	15
3.3	HBIN Blocks and Cells	17
3.4	Key Nodes (nk) and Value Keys (vk)	19
3.5	Subkey Lists (lf, lh, ri, li)	21
4	**SYSTEM FLOW AND ARCHITECTURE**	**24**
4.1	Overall System Flow	24
4.2	Component-Based Architecture	26
4.3	Data Flow Diagrams	28
5	**TECHNICAL IMPLEMENTATION - REGISTRY ENGINE**	**31**
5.1	Registry Engine (ntreg.cpp) Logic	31
5.2	Cell Level Access and Pointers	33
5.3	Enumerate Subkeys Algorithm	35
5.4	Value Key Retrieval Methods	37
6	**TECHNICAL IMPLEMENTATION - SAM MANAGER**	**39**

6.1	SAM Management (sam.cpp)	39
6.2	Path Resolution and Tree Walking	41
6.3	User Account Enumeration	43
6.4	Forensic Analysis of V and F records	45
7	**WIN32 UI ARCHITECTURE**	**48**
7.1	Dialog-Based Design Philosophy	48
7.2	Main Dialog Message Loop	50
7.3	ListView Data Binding	52
7.4	Automatic Path Detection Logic	54
8	**BUILD SYSTEM AND AUTOMATION**	**57**
8.1	MinGW Toolkit Integration	57
8.2	Static Linking and Portability	59
8.3	Resource Compilation with windres	61
9	**TROUBLESHOOTING AND FAQ**	**64**
9.1	Common Error Codes	64
9.2	Permission and Lock Exceptions	66
10	**RESULTS AND PERFORMANCE**	**69**
10.1	Functional Verification	69
10.2	Performance Benchmarks	71
11	**DISCUSSION AND SECURITY**	**74**
12	**CONCLUSION**	**76**
13	**FUTURE SCOPE**	**78**
14	**APPENDICES (FULL SOURCE CODE)**	**80**
14.1	Appendix A: ntreg.h	80
14.2	Appendix B: ntreg.cpp	82
14.3	Appendix C: sam.h	86
14.4	Appendix D: sam.cpp	88
14.5	Appendix E: main.cpp	90
14.6	Appendix F: crypto.h / crypto.cpp	94
14.7	Appendix G: resource.h / ntpwedit.rc	96

15	GLOSSARY AND REFERENCES	100

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Windows Registry is a hierarchical database that stores configuration settings and options on Microsoft Windows operating systems. It contains settings for the hardware, operating system software, most non-operating-system software, users, preferences of the PCs, etc. The registry provides a window into the operation of the runtime system.

One of the most critical parts of the registry is the **Security Accounts Manager (SAM)**. The SAM hive stores local user accounts, passwords (in hashed format), and group memberships. Under normal operating conditions, the SAM hive is locked by the Windows kernel (System process) and cannot be read or modified by standard user-mode applications.

The **"NTPWEdit Clone"** project is an endeavor to create a standalone, high-performance C++ application that can read and potentially modify these locked files by accessing them "offline." This means the tool can be used when the operating system is not running, or when the drive is mounted on another system.

1.2 Problem Statement

In the modern computing environment, access control is paramount. However, several scenarios arise where the standard access control mechanisms become a hindrance:

1. **Forgotten Credentials**: Users frequently forget local account passwords.
2. **Administrative Lockouts**: Security policies or malware can lead to the disablement of critical administrative accounts.
3. **Forensic Investigation**: Security professionals need to audit account activity on dead disks.
4. **Registry Corruption**: Standard OS tools often fail to handle slightly corrupted hives.

1.3 Objectives of the Project

1. **Develop a Robust Registry Parser**: Create a C++ engine capable of navigating binary hives (re-implementing `regf`, `hbin`, `nk`, `vk`, `lx` parsing).
2. **Bypass Kernel Locks**: Implement file-based access that works on non-active hives.
3. **Automate Identification**: Implement logic to automatically find the local system's SAM file.
4. **Enumerate Accounts**: Correctly extract user names and their corresponding Relative Identifiers (RIDs).
5. **Dependency-Free Build**: Ensure the tool can run on any Windows system (XP to 11).

CHAPTER 2: BACKGROUND AND LITERATURE SURVEY

2.1 Evolution of the Windows Registry

Before the Windows Registry, configuration settings were stored in numerous `.ini` files scattered across the system. This lead to "INI hell," where tracking settings was difficult, and performance suffered due to frequent text parsing. With the introduction of Windows NT, Microsoft introduced the Registry—a unified, binary, hierarchical database.

2.2 Windows Security Architecture History

Windows security has evolved through several iterations:

- **Windows NT 3.x/4.0**: Basic SAM-based security, LanMan (LM) hashes.
- **Windows 2000/XP**: Introduction of Active Directory and Kerberos, but local SAM remained for workgroups.
- **Windows Vista/7**: User Account Control (UAC) and mandatory integrity levels were added.
- **Windows 10/11**: Secure Boot, TPM-based protection, and Virtualization-based Security (VBS).

Despite these advancements, the local SAM remains the fallback for all local account authentication.

2.3 The Security Accounts Manager (SAM) Logic

The SAM is a database file in Windows that stores users' passwords. It is located at `%SystemRoot%\System32\config\SAM`. Passwords are never stored in plain text; instead, they are held as LM hashes or NT hashes. Windows uses a "SysKey" to encrypt these hashes before they even touch the disk, adding an extra layer of difficulty for recovery tools.

CHAPTER 3: REGISTRY HIVE FORMAT DEEP DIVE

3.1 Structural Overview

A registry hive is an on-disk file that contains a discrete body of registry keys, subkeys, and values. The format is a complex system of "Cells" and "Bins."

A Hive is organized into:

1. **Header Page**: The first 4096 bytes.
2. **Bins**: Large blocks (usually 4KB multiples) that contain cells.
3. **Cells**: The actual data units (Keys, Values, Lists).

3.2 The REGF Header

The `regf` header contains the magic "regf", sequence numbers, timestamps, and a root key offset. Our C++ structure `regf_header` maps these fields exactly to the binary layout.

The **Checksum** is calculated by XORing every 4-byte chunk of the header. If the calculation doesn't match the value stored in the header, the hive is considered corrupt by the system.

3.3 NK and VK Records

- **NK (Named Key)**: Represents a key. Includes name, timestamps, and pointers to subkeys and values.
- **VK (Value Key)**: Represents a value. Includes name, type, and data (or offset to data).

CHAPTER 5: TECHNICAL IMPLEMENTATION - REGISTRY ENGINE

5.1 Registry Engine (ntreg.cpp)

The implementation of the `Hive` class is where the majority of the low-level complexity resides.

5.1.1 Offsets and Addressing

Registry offsets are relative to the start of the first `hbin` block. Our `getOffset` function handles the conversion from these relative offsets to actual memory addresses in our buffer.

```
void* Hive::getOffset(DWORD offset) {
    if (offset >= data.size()) return nullptr;
    return &data[offset + 4]; // Skip the 4-byte block size
}
```

CHAPTER 14: APPENDICES (FULL SOURCE CODE)

14.1 Appendix A: ntreg.h

```
#ifndef NTREG_H
#define NTREG_H

#include <windows.h>
#include <vector>
#include <string>

#define REG_HEADER_SIZE 4096

#pragma pack(push, 1)
struct regf_header {
    char id[4]; // "regf"
    DWORD seq1;
    DWORD seq2;
    FILETIME mtime;
    DWORD ver_major;
    DWORD ver_minor;
    DWORD type;
    DWORD unknown1;
    DWORD root_key_offset;
    DWORD size;
    DWORD unknown2;
    char name[64];
    char unknown3[396];
    DWORD checksum;
};

struct nk_record {
    char id[2]; // "nk"
    WORD flags;
    FILETIME mtime;
    DWORD unknown1;
    DWORD parent_offset;
```

```
    DWORD subkey_count;
    DWORD unknown2;
    DWORD subkey_list_offset;
    DWORD unknown3;
    DWORD value_count;
    DWORD value_list_offset;
    DWORD sk_offset;
    DWORD class_offset;
    WORD name_len;
    WORD class_len;
    char name[1];
};

struct vk_record {
    char id[2]; // "vk"
    WORD name_len;
    DWORD data_len;
    DWORD data_offset;
    DWORD type;
    WORD flags;
    WORD unknown;
    char name[1];
};

struct subkey_entry {
    DWORD offset;
    char hash[4];
};

struct lx_record {
    char id[2]; // "lf", "lh", "ri", "li"
    WORD count;
    subkey_entry entries[1];
};
#pragma pack(pop)

class Hive {
public:
    Hive();
    ~Hive();
    bool open(const std::string& filename);
    bool save();
    void close();
    void* getOffset(DWORD offset);
    DWORD getRootOffset() { return header.root_key_offset; }
    DWORD findSubkey(DWORD parent_nk_offset, const std::string& name);
    std::vector<std::pair<std::string, DWORD>> enumerateSubkeys(DWORD parent_nk_offset);
    void* getValueData(DWORD nk_offset, const std::string& value_name, DWORD* out_type =
nullptr, DWORD* out_len = nullptr);
    bool writeBuffer(DWORD offset, void* buf, DWORD len);
    DWORD calculateChecksum();
```

```
private:
    std::string filepath;
    regf_header header;
    std::vector<char> data;
    bool is_dirty;
};

#endif
```

14.2 Appendix B: ntreg.cpp

```
#include "ntreg.h"
#include <fstream>
#include <iostream>

Hive::Hive() : is_dirty(false) {
    memset(&header, 0, sizeof(header));
}

Hive::~Hive() {
    close();
}

bool Hive::open(const std::string& filename) {
    filepath = filename;
    std::ifstream fs(filename, std::ios::binary | std::ios::ate);
    if (!fs.is_open()) return false;

    std::streamsize size = fs.tellg();
    if (size < REG_HEADER_SIZE) return false;

    fs.seekg(0, std::ios::beg);
    fs.read((char*)&header, sizeof(header));

    if (strncmp(header.id, "regf", 4) != 0) return false;

    data.resize((size_t)size - REG_HEADER_SIZE);
    fs.read(data.data(), data.size());

    return true;
}

bool Hive::save() {
    if (!is_dirty) return true;
    header.checksum = calculateChecksum();
    std::ofstream fs(filepath, std::ios::binary);
    if (!fs.is_open()) return false;
    fs.write((char*)&header, sizeof(header));
    fs.write(data.data(), data.size());
    is_dirty = false;
```

```
    return true;
}

void Hive::close() {
    if (is_dirty) save();
    data.clear();
    memset(&header, 0, sizeof(header));
}

void* Hive::getOffset(DWORD offset) {
    if (offset >= data.size()) return nullptr;
    return &data[offset + 4];
}

bool Hive::writeBuffer(DWORD offset, void* buf, DWORD len) {
    if (offset + len > data.size()) return false;
    memcpy(&data[offset], buf, len);
    is_dirty = true;
    return true;
}

DWORD Hive::findSubkey(DWORD parent_nk_offset, const std::string& name) {
    auto subkeys = enumerateSubkeys(parent_nk_offset);
    for (const auto& pair : subkeys) {
        if (_stricmp(pair.first.c_str(), name.c_str()) == 0) return pair.second;
    }
    return 0;
}

std::vector<std::pair<std::string, DWORD>> Hive::enumerateSubkeys(DWORD parent_nk_offset) {
    std::vector<std::pair<std::string, DWORD>> result;
    nk_record* nk = (nk_record*)getOffset(parent_nk_offset);
    if (!nk || strncmp(nk->id, "nk", 2) != 0) return result;
    if (nk->subkey_count == 0) return result;

    lx_record* lx = (lx_record*)getOffset(nk->subkey_list_offset);
    if (!lx) return result;

    if (strncmp(lx->id, "lf", 2) == 0 || strncmp(lx->id, "lh", 2) == 0) {
        for (int i = 0; i < lx->count; i++) {
            nk_record* sub_nk = (nk_record*)getOffset(lx->entries[i].offset);
            if (sub_nk && strncmp(sub_nk->id, "nk", 2) == 0) {
                std::string sub_name(sub_nk->name, sub_nk->name_len);
                result.push_back({sub_name, lx->entries[i].offset});
            }
        }
    } else if (strncmp(lx->id, "ri", 2) == 0 || strncmp(lx->id, "li", 2) == 0) {
        for (int i = 0; i < lx->count; i++) {
            lx_record* sub_lx = (lx_record*)getOffset(lx->entries[i].offset);
            if (sub_lx && (strncmp(sub_lx->id, "lf", 2) == 0 || strncmp(sub_lx->id, "lh", 2)
== 0)) {
                for (int j = 0; j < sub_lx->count; j++) {
```

```
                    nk_record* sub_nk = (nk_record*)getOffset(sub_lx->entries[j].offset);
                    if (sub_nk && strncmp(sub_nk->id, "nk", 2) == 0) {
                        std::string sub_name(sub_nk->name, sub_nk->name_len);
                        result.push_back({sub_name, sub_lx->entries[j].offset});
                    }
                }
            }
        }
    }
    return result;
}

void* Hive::getValueData(DWORD nk_offset, const std::string& value_name, DWORD* out_type,
DWORD* out_len) {
    nk_record* nk = (nk_record*)getOffset(nk_offset);
    if (!nk || strncmp(nk->id, "nk", 2) != 0 || nk->value_count == 0) return nullptr;
    DWORD* val_offsets = (DWORD*)getOffset(nk->value_list_offset);
    if (!val_offsets) return nullptr;
    for (DWORD i = 0; i < nk->value_count; i++) {
        vk_record* vk = (vk_record*)getOffset(val_offsets[i]);
        if (!vk || strncmp(vk->id, "vk", 2) != 0) continue;
        std::string v_name;
        if (vk->name_len > 0) v_name.assign(vk->name, vk->name_len);
        if (v_name == value_name || (value_name.empty() && vk->name_len == 0)) {
            if (out_type) *out_type = vk->type;
            if (out_len) *out_len = vk->data_len;
            if (vk->data_len & 0x80000000) return &vk->data_offset;
            return getOffset(vk->data_offset);
        }
    }
    return nullptr;
}

DWORD Hive::calculateChecksum() {
    DWORD sum = 0;
    DWORD* p = (DWORD*)&header;
    for (int i = 0; i < 127; i++) sum ^= p[i];
    return sum;
}
```

14.3 Appendix C: sam.h

```
#ifndef SAM_H
#define SAM_H

#include "ntreg.h"
#include <vector>
#include <string>

struct UserEntry {
```

```
    std::string username;
    DWORD rid;
    DWORD nk_offset;
};

class SAMManager {
public:
    SAMManager(Hive* hive);
    bool loadUsers();
    const std::vector<UserEntry>& getUsers() const { return users; }
    DWORD findUserNK(DWORD rid);

private:
    Hive* hive;
    std::vector<UserEntry> users;
};

#endif
```

14.4 Appendix D: sam.cpp

```
#include "sam.h"
#include <sstream>
#include <iomanip>

SAMManager::SAMManager(Hive* h) : hive(h) {}

bool SAMManager::loadUsers() {
    users.clear();
    DWORD sam = hive->findSubkey(hive->getRootOffset(), "SAM");
    if (!sam) sam = hive->getRootOffset();
    DWORD domains = hive->findSubkey(sam, "Domains");
    if (!domains) return false;
    DWORD account = hive->findSubkey(domains, "Account");
    if (!account) return false;
    DWORD users_key = hive->findSubkey(account, "Users");
    if (!users_key) return false;
    DWORD names = hive->findSubkey(users_key, "Names");
    if (!names) return false;
    auto name_keys = hive->enumerateSubkeys(names);
    for (const auto& pair : name_keys) {
        UserEntry entry;
        entry.username = pair.first;
        DWORD type = 0;
        hive->getValueData(pair.second, "", &type);
        entry.rid = type;
        entry.nk_offset = findUserNK(entry.rid);
        users.push_back(entry);
    }
    return !users.empty();
```

```
}

DWORD SAMManager::findUserNK(DWORD rid) {
    DWORD sam = hive->findSubkey(hive->getRootOffset(), "SAM");
    if (!sam) sam = hive->getRootOffset();
    DWORD domains = hive->findSubkey(sam, "Domains");
    DWORD account = hive->findSubkey(domains, "Account");
    DWORD users_key = hive->findSubkey(account, "Users");
    if (!users_key) return 0;
    char rid_hex[16];
    sprintf(rid_hex, "%08X", rid);
    return hive->findSubkey(users_key, rid_hex);
}
```

14.5 Appendix E: main.cpp

```
#include <windows.h>
#include <commctrl.h>
#include "resource.h"
#include "ntreg.h"
#include "sam.h"

Hive* g_hive = nullptr;
SAMManager* g_sam = nullptr;

INT_PTR CALLBACK DialogProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    switch (uMsg) {
    case WM_INITDIALOG:
        {
            HWND hList = GetDlgItem(hwnd, IDC_LIST_USERS);
            ListView_SetExtendedListViewStyle(hList, LVS_EX_FULLROWSELECT |
LVS_EX_GRIDLINES);
            LVCOLUMN lvc;
            lvc.mask = LVCF_TEXT | LVCF_WIDTH;
            lvc.pszText = (LPWSTR)L"User Name"; lvc.cx = 150; ListView_InsertColumn(hList,
0, &lvc);
            lvc.pszText = (LPWSTR)L"RID"; lvc.cx = 60; ListView_InsertColumn(hList, 1,
&lvc);
            lvc.pszText = (LPWSTR)L"Status"; lvc.cx = 80; ListView_InsertColumn(hList, 2,
&lvc);
            WCHAR sysDir[MAX_PATH];
            if (GetSystemDirectory(sysDir, MAX_PATH)) {
                std::wstring samPath = sysDir;
                samPath += L"\\config\\SAM";
                SetDlgItemText(hwnd, IDC_EDIT_PATH, samPath.c_str());
                PostMessage(hwnd, WM_COMMAND, MAKEWPARAM(IDC_BTN_OPEN, BN_CLICKED), 0);
            }
        }
        return (INT_PTR)TRUE;
```

```
    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDC_BTN_OPEN:
            {
                WCHAR path[MAX_PATH]; GetDlgItemText(hwnd, IDC_EDIT_PATH, path, MAX_PATH);
                if (g_hive) delete g_hive; if (g_sam) delete g_sam;
                g_hive = new Hive();
                char aPath[MAX_PATH]; WideCharToMultiByte(CP_ACP, 0, path, -1, aPath,
MAX_PATH, NULL, NULL);
                if (g_hive->open(aPath)) {
                    g_sam = new SAMManager(g_hive);
                    if (g_sam->loadUsers()) {
                        HWND hList = GetDlgItem(hwnd, IDC_LIST_USERS);
                        ListView_DeleteAllItems(hList);
                        const auto& users = g_sam->getUsers();
                        for (size_t i = 0; i < users.size(); ++i) {
                            LVITEM lvi = {0}; lvi.mask = LVIF_TEXT; lvi.iItem = (int)i;
                            WCHAR name[256]; MultiByteToWideChar(CP_ACP, 0,
users[i].username.c_str(), -1, name, 256);
                            lvi.pszText = name; ListView_InsertItem(hList, &lvi);
                            WCHAR rid[16]; swprintf(rid, 16, L"0x%X", users[i].rid);
                            ListView_SetItemText(hList, (int)i, 1, rid);
                            ListView_SetItemText(hList, (int)i, 2, (LPWSTR)L"Loaded");
                        }
                    } else MessageBox(hwnd, L"Failed to load users from SAM hive.",
L"Error", MB_ICONERROR);
                } else MessageBox(hwnd, L"Failed to open SAM hive file. Make sure you have
Administrator privileges.", L"Error", MB_ICONERROR);
            }
            return (INT_PTR)TRUE;
        case IDC_BTN_EXIT: EndDialog(hwnd, IDOK); return (INT_PTR)TRUE;
        }
        break;
    case WM_CLOSE: EndDialog(hwnd, IDCANCEL); return (INT_PTR)TRUE;
    }
    return (INT_PTR)FALSE;
}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int
nShowCmd) {
    InitCommonControls();
    return (int)DialogBox(hInstance, MAKEINTRESOURCE(IDD_MAIN_DIALOG), NULL, DialogProc);
}
```

GLOSSARY OF TERMS

- **ACB (Account Control Bits)**: A bitmask in the SAM hive that controls account properties like "Disabled" or "Locked Out."
- **Cell**: The smallest unit of data storage in a Registry hive (header + data).
- **Hive**: A file containing a discrete body of registry data.

- **NK (Named Key)**: A record type representing a registry key.
- **NT Hash**: An MD4-based hash used by Windows for user authentication.
- **RID (Relative Identifier)**: The last part of a SID that uniquely identifies an account within a domain.
- **SAM (Security Accounts Manager)**: The portion of the registry that manages user and group security.
- **VK (Value Key)**: A record type representing a registry value and its data.

REFERENCES

1. Microsoft Corporation, "Windows Registry Reference".
2. Russinovich, M., "Windows Internals".
3. Nordahl-Hagen, B., "Offline NT Password & Registry Editor".
4. RFC 1320, "The MD4 Message-Digest Algorithm".